THE STATE NIXOS WON'T MANAGE

Alexander Foremny, Applicative Systems

NixCon 2026, Krakow, Poland / 2026-09-27 09:00:00



Speaker

Alexander Foremny | github.com/aforemny

Nix/NixOS consultant | Hannover

Haskell ⟶ Nix | Mathematics (B. Sc) | 🦆🦆🎸🐟

Applicative Systems | applicative.systems

Nix/NixOS consulting for the public sector and private industry in the US and EU



NIXCADEMY
EXPERT EDUCATION

APPLICATIVE systems

Configuring KeyCloak and Forgejo

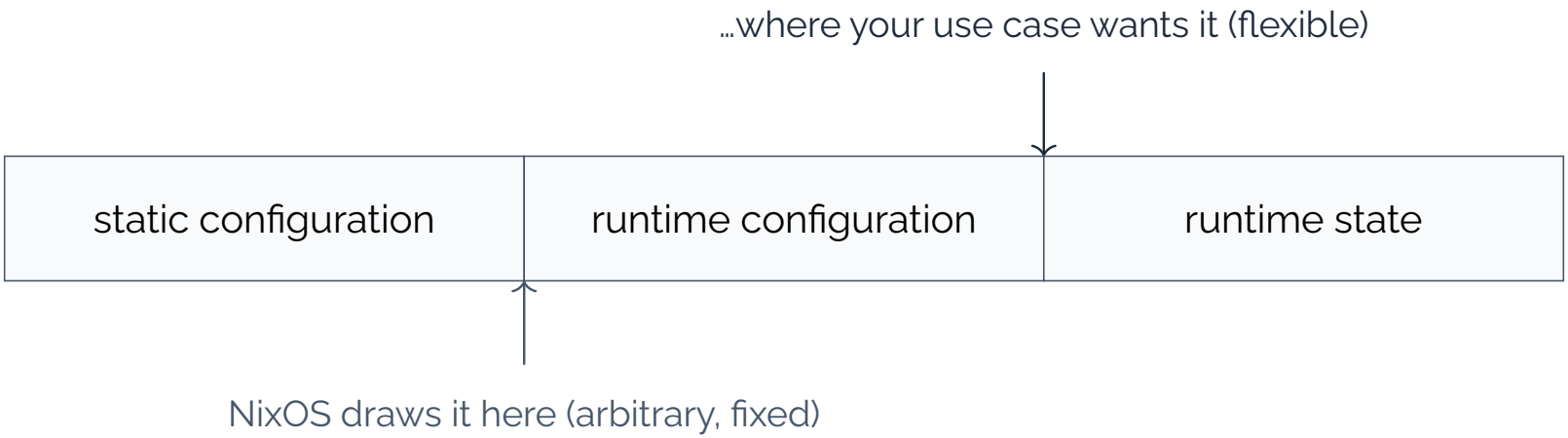
configuration.nix

```
{
  services.keycloak.enable = true;
  services.keycloak.settings.hostname = "id.example.com";
  services.forgejo.enable = true;
}
```

And now open a browser

```
#  1. log in as the temporary admin
#  2. create a realm
#  3. create the users, set their passwords
#  4. create an OpenID client, copy the client secret
#  5. paste that secret into your Forgejo configuration
```

The line is in the wrong place



Terraform already solves this

```
resource "keycloak_realm" "dunder_mifflin" {
  realm        = "dunder_mifflin"
  display_name = "Dunder Mifflin Paper Company"
}

resource "keycloak_user" "jhalpert" {
  realm_id = keycloak_realm.dunder_mifflin.id
  username = "jhalpert"
  email    = "jim@dundermifflin.com"
}

resource "keycloak_openid_client" "dunder-mifflin-infinity" {
  realm_id      = keycloak_realm.dunder_mifflin.id
  client_id     = "dunder-mifflin-infinity"
  client_secret = var.dunder_mifflin_app_client_secret
}
```

APP LICA TIVE

Streamline it into NixOS: declarative-runtime

```
{
  imports = [ flakeInputs.declarative-runtime.nixosModules.keycloak ];

  services.keycloak.runtime = {
    realms.dunder_mifflin.display_name = "Dunder Mifflin Paper Company";
    users.jhalpert = {
      realm = "dunder_mifflin";
      email = "jim@dundermifflin.com";
    };
    openid_clients.dunder-mifflin-infinity = {
      realm = "dunder_mifflin";
      client_secretFile = "/etc/secrets/dunder-mifflin-app-client-secret";
    };
  };
}
```

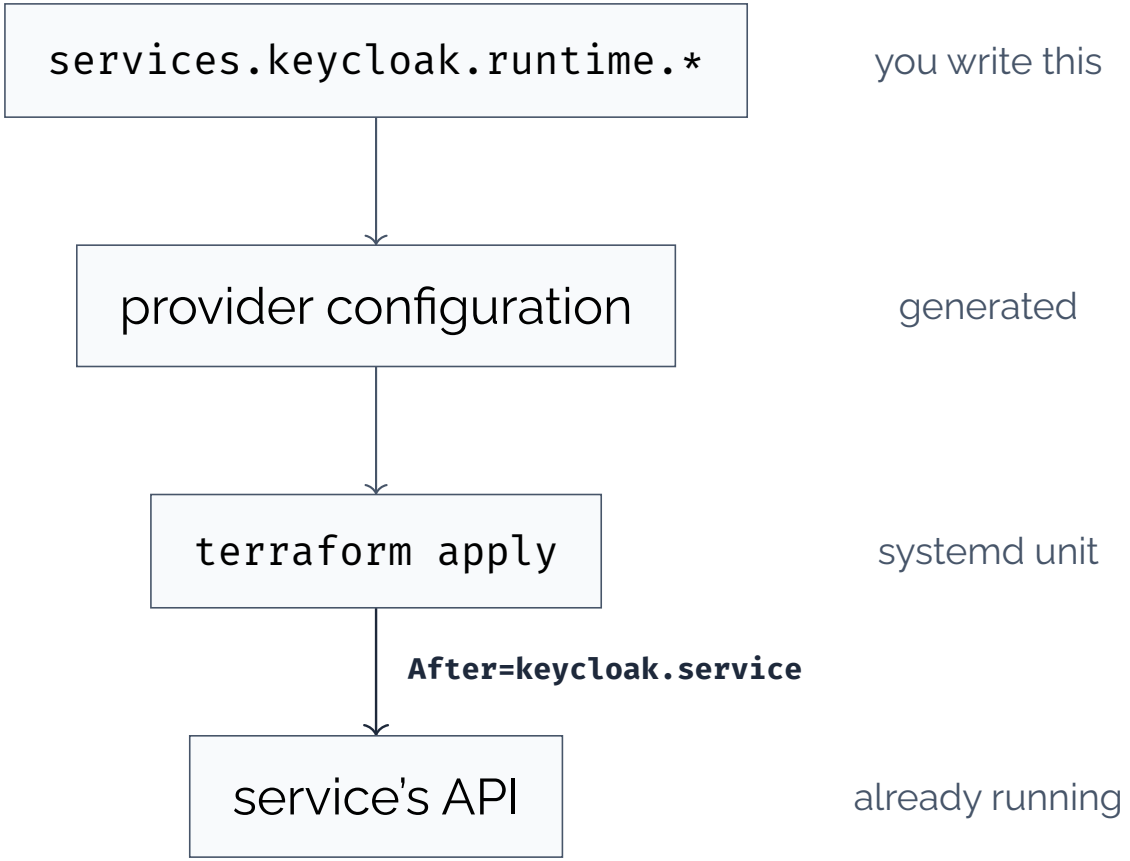
APP LICA TIVE

Demo: two users, one private repository

```
nix run github:applicative-systems/declarative-runtime#keycloak-forgejo
```

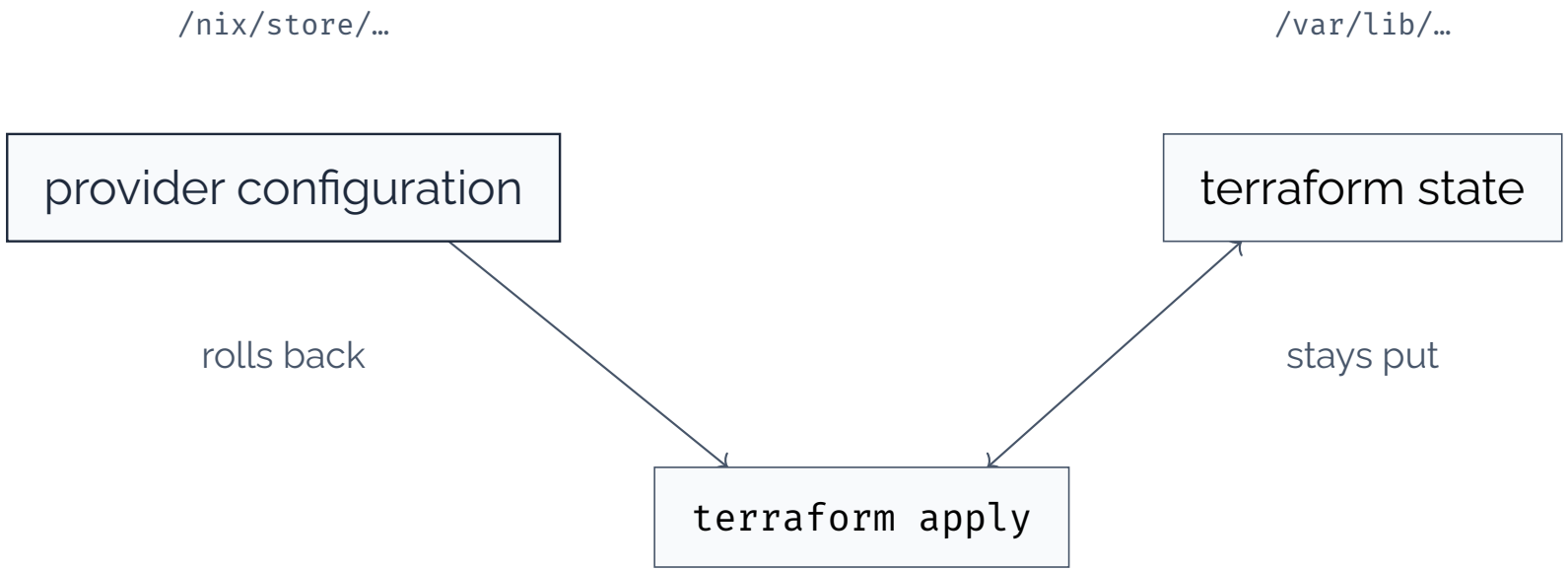
IMPLEMENTATION

NixOS calls Terraform for you



APP LICA TIVE

Terraform only manages what it created



Removing resources works

```
 services.keycloak.runtime.realms = {
-  acme = { ... };
   intranet = { ... };
 };
```



```
Apply complete! Resources: 0 added, 0 changed, 1 destroyed.
```

APP LICA TIVE

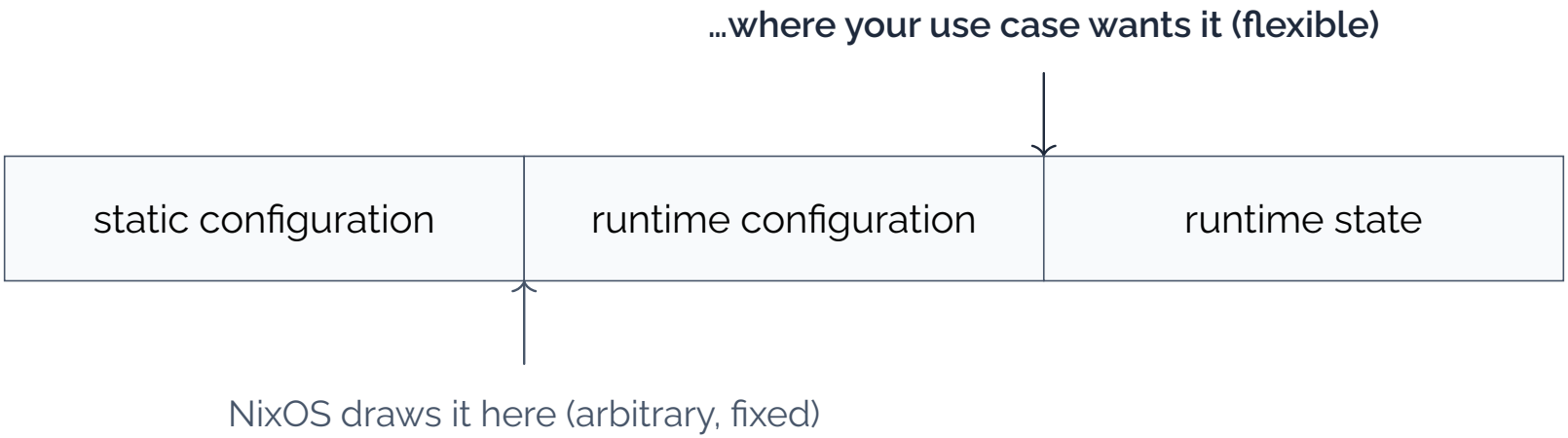
Rollback works

```
$ nixos-rebuild switch --rollback
```



```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

You draw the line



Terraform is activation based

```
$ systemctl list-timers 'keycloak*'
```



```
0 timers listed.
```

APP LICA TIVE

Repository: declarative-runtime

Service	Provider	Resources	Options
Keycloak	`keycloak/keycloak 5.9.0`	109 of 109	879
Forgejo	`svalabs/forgejo 1.5.0`	15 of 15	126

Closing: Spend less time in your browser

😐

BOOTSTRAPPING REQUIRES A BROWSER

😕

DECLARATIVITY OF SERVICES WON'T CHANGE

🤩

DECLARATIVE-RUNTIME ENABLES YOUR USE-CASE



github:applicative-systems/declarative-runtime

APPLICATIVE

Appendix

- Planned feature: gradual adoption
- Authentication credentials
- How to support a service
- Isn't this just terranix?
- What this could become

Planned feature: gradual adoption

```
 services.keycloak.runtime.realms = {
+  acme = { ... };  # a realm your operators already made
 };
```



```
keycloak_realm.acme: Importing... [id=acme]
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

APP LICA TIVE

Authentiation credentials

Problem

- Terraform requires API access
- Credentials should not require a browser
- ⟶ credentials should not have to be configured

Principle

- local privileged user ⟶ credentials

KeyCloak

1. admin recovery: local creation of temporary admin service account
2. creation of permanent Terraform client credentials
3. removal of recovery account

APP LICA TIVE

How to support a service

- Find Terraform provider
- Figure out local credential creation
  - or accept your fate: credentials have to be configured
  - (⟶ file feature request upstream)
- Imitate existing pairing (uses `./lib`)

APP LICA TIVE

Isn't this just terranix?

terranix

Nix as a *frontend* to Terraform

You run `terraform apply`

Targets cloud resources (VMs)

declarative-runtime

Terraform as a *backend* to NixOS.

You run `nixos-rebuild`

Targets the services on a host

What this could become

Immediately: bit of extra configurability that makes your bootstrapping or automation actually work.

Eventually: if services expose their runtime, it could become meaningful to write abstract service integrations which you would only instantiate.

```
{
  services.forgejo.single-sign-on.keycloak.enable = true;
  # or services.forgejo.single-sign-on.authelia.enable = true;
  # or ..
}
```